

Backend Engineer Interview Guide

Overview

Our backend technical interview is a collaborative, hands-on coding session conducted live over Zoom using CoderPad. We'll focus on your reasoning about execution flow, data propagation, and clean system design as complexity increases. You don't need to "one-shot" the problem, we're more interested in how you problem-solve.

Our best candidates are strong collaborators who ask clarifying questions early. They make reasonable assumptions, and clearly articulate them to their interviewer. They craft reliable, structured code. They don't optimize prematurely, instead opting to start simple and iterate.

What to expect:

Your interviewer will spend a few minutes on introductions, then dive right into the problem. You'll spend about an hour working through the problem - you can collaborate with your interviewer and ask clarifying questions throughout the interview! They'll leave about 10 minutes at the end of your interview for discussion and any additional questions.

- You'll be given a workflow-style system to reason about
- You'll write real, working code. You may use Python or TypeScript.
- The problem will evolve incrementally - you'll extend your solution as new structural concepts are introduced
- Your interviewer will act as a collaborator and can answer clarifying questions

What we're looking for:

- Technical & Systems Skills
 - Your ability to model execution flow and data availability
 - Your ability to clarify your assumptions about state & scope
 - Creating clean, extensible code structure
 - Building an extensible solution to avoid massive rewrites as requirements are added
 - Identifying and explaining tradeoffs in data structures & control flow
- Problem Solving & Communication
 - Clearly explaining your reasoning as you go and asking clarifying questions
 - Iterating toward a cleaner solution
 - Anticipating and confidently handling edge cases
 - Refactoring thoughtfully as requirements evolve

Things to Know:

You'll find these skills helpful:

- Traversing hierarchical or graph-like structures
- Writing functions that compute derived state from structured inputs
- Modeling scoped data (what's available where and why)
- Extending an initial solution as new constraints are introduced
- Keeping code modular and readable under evolving requirements

You'll find these concepts helpful:

- Working with trees or graph-like data structures
- Reasoning about state machines or execution contexts
- Understanding of variable scope and data propagation
- Basic familiarity with recursion or traversal patterns

You don't need to memorize anything. Your ability to confidently reason through structured systems is what matters most. You also won't need any external libraries.